

Cloud computing is the on-demand delivery of IT resources—including servers, storage, databases, and software—over the internet with pay-as-you-go pricing.

NOTE: STUDY PPT1 AND PPT2 FIRST

1. About the Jericho Forum

- The **Jericho Forum** is part of **The Open Group**.
 - Its main focus is **how to protect cloud networks** and ensure **secure collaboration** in cloud environments.
 - They created the **Cloud Cube Model**, which helps organizations **understand different types of cloud setups** and their **security boundaries**.
 - Reference paper: *“Cloud Cube Model: Selecting Cloud Formations for Secure Collaboration.”*
-

2. Purpose of the Cloud Cube Model

- It helps classify cloud networks based on **four dimensions (factors)**.
 - It shows how **different types of cloud configurations** affect where the **boundary between a client’s network and the cloud** begins and ends.
 - It also explains that the **old idea of a firewall defining the network boundary is outdated** in cloud computing.
-

3. Four Dimensions of the Cloud Cube Model

(a) Physical Location of the Data: Internal (I) / External (E)

- **Internal (I):** Data and systems are located *within* the organization’s infrastructure.
 - **External (E):** Data and systems are located *outside* the organization, such as in a public cloud.
 - 🖐️ This defines the **organization’s boundaries**.
-

(b) Ownership: Proprietary (P) / Open (O)

- **Proprietary (P):** The organization owns or uses closed systems — limited interoperability, vendor lock-in possible.
 - **Open (O):** Based on open standards — easier interoperability, flexibility, and data portability.
 - 🖐️ This shows who **owns the technology** and how easily systems can connect or migrate.
-

(c) Security Boundary: Perimeterised (Per) / De-perimeterised (D-p)

- **Perimeterised (Per):** Operations and data remain *inside* traditional network security boundaries (like firewalls).
 - **De-perimeterised (D-p):** Operations occur *outside* traditional boundaries — the cloud blurs or removes the concept of a single secure perimeter.
 - 🖐️ Measures **where the security controls exist** — inside or outside the firewall.
-

(d) Sourcing: Insourced / Outsourced

- **Insourced:** Services are provided and managed *by the organization itself*.
 - **Outsourced:** Services are delivered *by an external cloud provider*.
 - 🖐️ Identifies **who delivers or manages** the cloud service.
-

4. The Eight Possible Cloud Forms

- The four dimensions can combine to form **eight types of cloud setups**.
 - Based on the **security boundary** dimension, there are two groups:
 - **Perimeterised (Per):** IP, IO, EP, EO
 - **De-perimeterised (D-p):** IP, IO, EP, EO
(I = Internal, E = External, P = Proprietary, O = Open)
-

5. What the Model Shows

- The **traditional network boundary** (defined by firewalls) **no longer applies** in cloud computing.
 - In the cloud, **data, apps, and services** may exist **across multiple locations, ownership models, and security zones**.
 - Therefore, organizations must rethink **security policies** to handle **shared and dynamic environments**.
-

In summary:

The **Cloud Cube Model** helps organizations understand *where* their data lives, *who owns* the technology, *how security* is managed, and *who provides* the service — all crucial for making **secure and well-informed cloud adoption decisions**.

Here's a **simple, point-by-point explanation** of the entire topic — **"Assessing the Role of Open Standards in Cloud Computing"** 🖐️

1. What are Open Standards?

- **Open standards** define how data is **formatted, stored, and exchanged**, allowing different systems and organizations to **communicate easily**.
 - They are **publicly available** and developed through **collaboration and consensus**.
 - The goal is to **enable interoperability** (systems working together) and **data exchange** across platforms.
 - Open standards help prevent **vendor lock-in** (being stuck with one company's technology).
 - They **promote compatibility, interoperability, and innovation**.
-

2. Importance of Open Standards in Cloud Computing

- Cloud environments need **interoperability**, so open standards are essential.
 - Interoperability can be of two types:
 - **Horizontal**: Between different applications.
 - **Vertical**: Within components of the same application.
 - Without open standards, companies may create **proprietary or isolated cloud systems** (known as “*stovepipe clouds*”), which limit flexibility.
 - Clients want the **freedom to move data or apps** between cloud providers — so there's a strong industry push for **standard-based clouds**.
-

3. Architectural Standards Used in Cloud Computing

The cloud industry follows several **architectural and technical standards** to ensure compatibility:

1. **Platform virtualization of resources** – allows multiple virtual systems on one machine.
2. **Service-oriented architecture (SOA)** – builds cloud services as independent, reusable components.
3. **Web-application frameworks** – standardize how web apps are built and deployed.
4. **Deployment of open-source software** – encourages transparency and collaboration.
5. **Standardized web services** – ensures that different applications can communicate over the web.
6. **Autonomic systems** – systems that manage themselves automatically.
7. **Grid computing** – links multiple computers to work together on shared tasks.

 These standards support **cloud business models** like:

- **SaaS (Software as a Service)**
- **Web 2.0 applications**

- **Utility computing (pay-as-you-go services)**

They ensure **data portability** and **universal accessibility**.

4. Industry Push for Open Cloud Platforms

- There's a **race to develop open cloud platforms** that can compete with proprietary systems like:
 - **Microsoft Azure**
 - **VMware vSphere**
 - Companies and organizations are developing **open-source alternatives** to keep cloud ecosystems open and flexible.
-

5. Example 1 – OpenStack Project

- **Rackspace** (a large IaaS provider) launched **OpenStack** in **July 2010**.
 - It started using code from **Rackspace Cloud Files** and **Cloud Servers** technologies.
 - **NASA** contributed its **Nebula Cloud Platform** technology.
 - The project's code was released under the **Apache 2.0 open-source license**.
 - **Founding members** included: AMD, Citrix, Dell, Intel, NTT Data, and others.
 - **OpenStack.org** is the official website for this open cloud platform.
-

6. Example 2 – Eucalyptus Project

- **Eucalyptus** stands for "*Elastic Utility Computing Architecture for Linking Your Programs to Useful Systems.*"
 - It's a **Linux-based software platform** for building **IaaS (Infrastructure as a Service)** clouds.
 - It allows creation of **private or hybrid clouds** based on computer clusters.
 - **Compatible with Amazon Web Services (AWS)** — connects with **EC2** and **S3**.
 - Developers can use it as a **sandbox** for testing cloud applications.
 - Works with major **virtualization technologies**: VMware, Xen, and KVM.
 - Originated at the **University of California, Santa Barbara** by **Rich Wolski**.
 - **Eucalyptus Systems** company was formed in **2009** for commercial support.
 - Supported by major **Linux vendors** (like Ubuntu, Red Hat, etc.).
-

7. Other Open Cloud Projects and Research

- OpenStack and Eucalyptus are **not the only projects** — many other open-source and research-based cloud initiatives exist.
 - The **IEEE Technical Committee on Services Computing (TCSC)** supports work in this field.
 - Website: <http://tab.computer.org/tcsc/>
 - They organize the **CLOUD conference** and publish research on cloud standards and interoperability.
-

8. OpenStack's Key Deliverables

OpenStack's first two major components were:

1. **Distributed Object Store** – based on **Rackspace Cloud Files** (used for storing large amounts of unstructured data).
2. **Scalable Machine Provisioning** – based on **NASA Nebula** and **Rackspace Cloud Servers** (used to create and manage virtual servers).

OpenStack consists of:

- **OpenStack Compute:** Software to automatically create and manage **virtual private servers** on standard hardware.
 - **OpenStack Storage:** Software that provides **redundant, object-based storage** using clusters of commodity servers.
-

9. Summary of Key Takeaways

- **Open standards** make cloud systems compatible, flexible, and interoperable.
- They **eliminate vendor lock-in**, ensuring customers can switch providers easily.
- **OpenStack** and **Eucalyptus** are key open-source platforms promoting open standards in the cloud.
- Open standards are essential for the **growth, scalability, and collaboration** of the cloud industry.
- The **traditional proprietary systems** are being challenged by **open-source, standards-based cloud ecosystems**.

1. Cloud Computing Ecosystem – Four Layers

Cloud computing is built on **four layers**, each representing a different service model:

1. **Application Layer → SaaS (Software as a Service)**

- Provides ready-to-use applications over the internet.
- Example: Gmail, Salesforce, Microsoft 365.

2. Platform Layer → PaaS (Platform as a Service)

- Provides platforms for developers to build, test, and deploy applications.
- Example: Google App Engine, Heroku.

3. Infrastructure Layer → IaaS (Infrastructure as a Service)

- Provides virtualized computing resources such as servers, storage, and networking.
- You pay only for what you use — a **utility model** (like electricity).
- Offers **scalability** and flexibility, allowing small companies to grow quickly.
- This model has encouraged many **start-ups**, even during economic recessions.

4. Hardware Layer

- The physical foundation: data centers, physical servers, and networking equipment.

2. What Is Infrastructure as a Service (IaaS)?

- **IaaS** virtualizes **hardware resources** (servers, storage, and networking) in the cloud.
- The **service provider** owns and manages all physical infrastructure.
- Developers create **virtual machines (VMs)** and use them as if they were real computers.
- Users can provision:
 - Virtual servers
 - Virtual storage
 - Virtual networks
 - Other computing components on demand
- **Mapping:** Virtual resources are connected (mapped) to physical servers in the data center.
- When a client requests resources, the IaaS system redirects those requests to real hardware performing the work.

3. Workloads in IaaS

- A **workload** = the fundamental virtual client unit in an IaaS environment.
- It represents how much work a physical or virtual server can perform.
- Measured using metrics such as:
 - **Transactions Per Minute (TPM)**

- **Disk I/O operations per second (IOPS)**
 - **RAM usage**
 - **Network throughput and latency**
 - Each workload may require a **small, medium, or large** server instance depending on performance needs.
 - In cloud computing, customers **reserve virtual instances** matching these workloads.
 - The IaaS system allocates compute, storage (RAID), and network resources accordingly.
-

4. Logical Structure of IaaS

- Physical systems are divided into **logical units**:
 - **LUNs (Logical Unit Numbers)**: Logical storage blocks.
 - **Cloud interconnect layer**: Virtual network layer assigning IPs from a shared pool.
 - **Virtual application software layer**: Runs applications inside virtual machines.
 - Clients in IaaS are usually given their **own private network**.
-

5. Example: Amazon EC2 (Elastic Compute Cloud)

- Each EC2 instance behaves like a **separate network**.
 - You can create a **Virtual Private Cloud (VPC)** to connect instances securely.
 - When scaling EC2, each added instance creates a new logical network — this improves scalability but adds **network overhead** due to routing between networks.
-

6. Networking in IaaS

- **Amazon Web Services (AWS)**:
 - Restricts **broadcast** and **multicast traffic** because **Layer-2 networking** (Data Link Layer) isn't supported.
 - **Rackspace Cloud**:
 - Uses a similar IP assignment model to AWS.
 - **Cloudscaling.com** and **VMware clouds**:
 - Provide **Level-2 networking** (supports VLANs through tunneling protocols).
 - **Common Level-2 protocols**: VLAN tunneling methods that enable private sub-networks.
-

7. Example – eCommerce System on IaaS

A typical **transactional eCommerce** stack may include:

- Web Server
- Application Server
- File Server
- Database
- Transaction Engine

Each component runs as a **separate workload**, handling tasks like:

- Querying the database
 - Processing business logic
 - Serving web pages to users
-

8. Classic Example – Amazon Web Services (AWS)

- AWS uses **data centers** running **Xen virtualization**.
 - Physical servers are divided into **logical compute units** of various sizes.
 - Developers can:
 - Apply predefined **system images** or custom ones.
 - Create **databases, partitions, and messaging services** to support distributed apps.
 - AWS uses a **Service-Oriented Architecture (SOA)** model for IaaS.
-

9. Pods, Aggregation, and Sizing

- Each instance (VM) can support only a limited number of users (a **Pod**).
- When the user limit is reached, instances are **cloned** to create more Pods.
- **Pod**: A group of users within a particular instance.
- Managed by a **Cloud Control System (CCS)** — in AWS, this is the **AWS Management Console**.
- **Pods** are combined into larger **pools** within a **region** called an **Availability Zone**.
- **Zones** are physical data centers in locations like:
 - Northern California
 - Northern Virginia
 - Ireland
 - Singapore

If one zone fails, **failover systems** automatically switch to another, ensuring **high availability**.

10. Silos in Cloud Infrastructure

- When different private clouds or systems **cannot communicate or interoperate**, they form **silos**.
 - **Silos = isolated computing environments** (like islands).
 - They often appear when private virtual networks are created inside IaaS.
 - **Drawbacks:**
 - Reduced **interoperability**.
 - Potential **vendor lock-in**.
 - **Benefits:**
 - Higher **security** and **data protection** for sensitive applications.
 - Silos are most common in **PaaS** models (e.g., Force.com, QuickBase) but also occur in IaaS setups.
-

11. Summary

- Cloud computing has **four main layers**: Application (SaaS), Platform (PaaS), Infrastructure (IaaS), and Hardware.
- **IaaS** provides on-demand virtualized hardware resources — pay only for what you use.
- **Workloads** define how much computing power a virtual machine requires.
- **Logical components** like LUNs, virtual networks, and software layers create flexible cloud environments.
- **AWS EC2** is the classic example of IaaS in action.
- **Pods, zones, and silos** describe how workloads are organized, scaled, and isolated.
- IaaS enables **scalability**, **cost efficiency**, and **high availability**, but careful design is needed to avoid **network overhead** and **vendor lock-in**.

Platform as a Service (PaaS) – Overview

1. **Meaning:**
 - PaaS means *Platform as a Service*.
 - It's a **cloud computing model** where a third-party provider gives users a **complete platform** to **develop, run, and manage applications**.

- You don't have to manage servers, storage, or operating systems — just focus on building your app.
2. **Purpose:**
- Offers a **ready-to-use environment** for developers.
 - Enables delivery of both **simple apps** and **large enterprise applications**.
3. **Main Idea:**
- The platform provides **development tools, runtime environment, databases, and hosting** — everything you need to build apps without handling hardware.
4. **Developer Use:**
- Developers use the platform's **built-in tools** to create custom solutions.
 - These tools may support specific **languages, frameworks, or APIs**.
5. **Integration:**
- PaaS is a **fully integrated development environment (IDE)** in the cloud.
 - It supports common standards like **HTML, JavaScript, and rich media technologies** for UI development.
-

How PaaS Works

6. **User Role:**
- Customers interact with the software to **enter/retrieve data, perform actions, or customize** features (depending on permissions).
7. **Vendor Role:**
- The vendor handles everything: **hardware, software, maintenance, and lifecycle management**.
 - Users only use the platform — they don't manage infrastructure.
8. **Examples:**
- **Google App Engine:** Developers use Google's APIs and predefined file/data structures.
 - **Force.com (Salesforce):** Provides tools for developers to build add-ons for Salesforce apps.
9. **Add-on Development:**
- Developers can create apps in languages like **Python** using a platform's API (e.g., Google's).
 - The platform vendor (e.g., Google) also provides many **web services** under the same model — like **Gmail, Google Maps, Google Earth**, etc.

10. Vendor Lock-in Issue:

- PaaS can cause **vendor lock-in**, meaning apps built on one platform may not run elsewhere.
 - Example: An app built for Google App Engine using Google APIs will only work in Google's environment.
-



Benefits of PaaS

RMS MFC

11. Reduced Coding Time:

- Comes with **pre-coded components** like workflow, authentication, search, etc., reducing manual coding.

12. More Development Power:

- Offers advanced **tools and services** that increase development capability without hiring more developers.

13. Supports Remote Teams:

- Being cloud-based, it allows **real-time collaboration** among developers working from different locations.

14. Multi-Platform Development:

- Enables development for **multiple devices** — web, mobile, or desktop.

15. Full App Lifecycle Support:

- Provides all tools needed to **build, test (in virtual machines), deploy, and update** apps — all in one place.

16. Cost-Effective:

- No need to buy hardware or start from scratch.
 - Saves money on **equipment** and increases **productivity**.
-



Business Impact

17. Encourages Innovation:

- PaaS drives **business innovation** by simplifying app development and reducing infrastructure complexity.

18. Supports Digital Transformation:

- Helps businesses **grow and modernize** by moving to cloud-based systems.

19. Key Business Benefits:

- Offers **flexibility, productivity, secure data storage, advanced tools, and streamlined operations**.

20. Overall:

- PaaS allows companies to focus on **developing great applications**, while the provider manages everything else behind the scenes.

Software as a Service (SaaS) – Overview

1. Meaning:

- SaaS stands for **Software as a Service**.
- It's a **cloud-based model** where software applications are delivered to users **over the Internet**.

2. Purpose:

- SaaS allows users to **access software online** without installing it on their local computer.
- Everything runs on the cloud, and you just use it via a **web browser**.

3. Common Examples:

- Email and office tools such as **Gmail, Google Calendar, Microsoft Office 365, Zoho Office Suite**, and **QuickBooks Online**.

4. Payment Model:

- SaaS is **pay-as-you-go** — you pay only for how long or how much you use.

Key Features of SaaS

5. Complete Service Model:

- SaaS provides the **entire stack** — hardware, software, and complete solution — as a **ready-to-use service**.

6. Access:

- Applications are **hosted online** and accessible **anywhere, anytime**, usually through a browser.

7. User Simplicity:

- Users only interact with the software; all **backend details** like servers, updates, and maintenance are handled by the provider.

8. Familiarity:

- Most users already use SaaS tools daily — for email, accounting, HR management, CRM, or help-desk support.

Customization and APIs

9. Customizability:

- Some SaaS applications (like office suites) offer **limited customization**, allowing only changes to **preferences or options**.

10. APIs for Developers:

- Advanced SaaS products expose **Application Programming Interfaces (APIs)** so developers can:
 - Modify **security models**
 - Change **data schema or workflows**
 - Build **custom applications** using the service

11. Examples:

- **Salesforce.com** and **Quicken.com** are SaaS platforms with open APIs for customization.
- So SaaS is **not always fixed or static** — it can be extended by developers.

Characteristics of SaaS

12. Global Access:

- Available worldwide through the **Internet** via browsers.

13. Subscription/Usage-Based License:

- Users pay a **recurring fee** (monthly or yearly) or per-use basis.

14. Vendor Maintenance:

- The **service provider** handles all updates, maintenance, and monitoring.

15. Client-Side Code:

- There might be small executable code on the user's device, but users **don't manage or update** it.

16. Low Cost:

- Reduced **distribution, installation, and maintenance** costs.
- Usually **cheaper** than traditional, locally installed software.

17. Automatic Updates:

- Updates, patches, and new versions are deployed **automatically** by the vendor.

18. Scalability & Accessibility:

- SaaS applications can easily **scale** with user demand and are **accessible anywhere**.

19. Same Version for All Users:

- Everyone uses the **same software version**, ensuring compatibility across users.

20. Multi-Tenancy Support:

- One software instance serves **multiple users**, each with separate data and configurations.

21. Alternative:

- Some SaaS systems use **virtualized individual instances** per user, but this is **less common**.
-

Benefits of SaaS

22. No Software Installation or Updates:

- Users don't need to manually install or update software — everything happens automatically online.

23. No Licensing or Purchase Costs:

- You pay **only for usage**; no need to buy full software licenses.

24. No Server Costs:

- Since everything runs in the cloud, there's **no need to buy or maintain servers**.

25. Pay-As-You-Use:

- Flexible payment — pay for what you use, when you use it.

26. Accessibility:

- Access your data **anytime, anywhere** from any Internet-connected device.

27. No IT Maintenance Needed:

- No need for a dedicated **IT team** or maintenance staff.

28. No Security Budget:

- The vendor handles all **security measures**, reducing user costs.

29. Data Backup & Reliability:

- Data is **backed up automatically** at regular intervals to prevent data loss.

30. Cluster Infrastructure (High Availability):

- Cloud servers use **clusters** — multiple redundant computers.
 - If one fails, another takes over instantly, ensuring **continuous service without interruption**.
-

In summary:

- **SaaS = Ready-to-use software over the Internet.**

- The provider manages **everything** — servers, updates, security, and maintenance.
- You just **use the app, pay per use**, and **access it anywhere** with **zero setup**.



Open SaaS (Open Source + SaaS)

1. **Definition:**
 - SaaS (Software as a Service) built using open-source software is called **Open SaaS**.
2. **Advantages of Open Source Software in SaaS:**
 - **Cheaper deployment:** No need to buy OS or software licenses.
 - **Less vendor lock-in:** You're not tied to one company.
 - **More portability:** Easier to move applications between systems.
 - **Wider support:** Many developers are already trained in open-source tools.
3. **Popular Open Source Tools Used in SaaS:**
 - **Linux, Apache, MySQL, Perl** — together called the **LAMP platform**.
4. **Business Impact:**
 - **Better profitability:** Lower development costs.
 - **More robust solutions:** Stronger, more reliable systems due to community support.

Open SaaS and SOA (Service-Oriented Architecture)

5. **Component-based design:**
 - SaaS applications are made of reusable components that support **mashups**.
6. **What is a Mashup?**
 - A web application that **combines data and features from multiple sources** into one interface.
 - Example: Showing restaurant data on **Google Maps**.
7. **Why Mashups Matter:**
 - They are key examples of **Web 2.0**, promoting **social networking and interactivity**.
8. **Three Components Required for a Mashup:**
 - **User Interface:** Created using HTML/XHTML, Ajax, JavaScript, or CSS.

- **Web Services:** Accessed via APIs and protocols like **SOAP**, **REST**, **XML/HTTP**, **XML-RPC**, **JSON-RPC**.
 - **Data Transfer Formats:** XML, **KML** (Keyhole Markup Language), **JSON**, etc.
-

Open Mashup Alliance (OMA)

9. What It Is:

- A **non-profit industry group** supporting enterprise mashup technologies.
- Website: www.openmashup.org

10. Standard Supported:

- Developing a new standard called **Enterprise Mashup Markup Language (EMML)** — a **Domain Specific Language (DSL)** for mashups.

11. Growth Prediction:

- The use of mashups is expected to **grow 10x within a few years**.

12. Gartner's Prediction:

- By **2011**, around **25% of all software** would be delivered as **SaaS**, either directly by vendors or through **aggregators**.

13. What Is an Aggregator?

- A company that **bundles SaaS apps from different vendors** and offers them as a **single unified solution**.
-

Salesforce.com and CRM SaaS

14. Example of SaaS:

- **Salesforce.com** — a leader in **Customer Relationship Management (CRM)** SaaS.

15. Features Offered by Salesforce SaaS:

- Sales, service, support, marketing, content management, analytics, and team collaboration via **Chatter**.

16. Background:

- Founded in **1999** by former **Oracle executives**.
- Early adopters of cloud computing technologies.

17. From SaaS to PaaS:

- Salesforce expanded to **Force.com**, allowing developers to create **custom apps** on its platform.

18. Development Platform Details:

- Uses a Java-like language called **Apex**.
- Interfaces are built with **XML**, **HTML**, **Ajax**, and **Flex**.

19. Ecosystem Growth:

- Now hosts **nearly 1,000 applications** from **hundreds of vendors**.

Identity as a Service (IDaaS)

◆ What is IDaaS?

1. Definition:

- IDaaS stands for **Identity as a Service** — it provides **cloud-based authentication** or **identity management** to organizations that subscribe to it.
- In simple words: It helps verify “who you are” before giving access to systems, apps, or data.

2. X-as-a-Service model:

- “X” can be anything (software, platform, identity, etc.).
- It means a feature or service is **delivered remotely over the internet** by a **third-party provider**, instead of being handled locally by in-house staff.

3. Example:

- Local email → Microsoft Outlook (installed on your PC).
- Cloud email → Gmail (accessed through the internet).
- Similarly, identity/security can be provided “as a service” in the cloud.

4. Goal of IDaaS:

- To **confirm user identity** and **grant correct access** to files, software, or resources **at the right time**.
- Without IDaaS, companies must maintain all identity systems themselves, which is time-consuming and complex.

◆ Why Use Cloud-based Identity Services?

5. Handling BYOD (Bring Your Own Device):

- Employees use different devices (phones, tablets, laptops).
- Local systems must adapt quickly to each device — but with cloud IDaaS, this adaptation is automatic and centralized.

6. Simpler Management:

- Cloud-based identity services are managed by **experts** who already solved identity problems for many organizations.

- This saves time, reduces errors, and ensures security.

7. Not a New Concept:

- Digital identity management has existed since early networking days.
 - “IDaaS” is just a **modern term (FLAV – Four Letter Acronym)** for services that have long existed in some form.
-

DNS as an Example of IDaaS

8. Domain Name Service (DNS):

- DNS can run privately or on the internet.
- It is a core **identity service of the Internet** that links **domain names to IP addresses**.
- Example: The domain .com or .org servers act as IDaaS servers.

9. How DNS establishes identity:

- It connects a domain name to:
 - Assigned IP addresses
 - The domain owner’s info
 - Other metadata
 - The domain name = identity; IP and metadata = properties of that identity.
-

◆ Understanding “Identity” in Computing

10. Meaning of identity:

- A **set of characteristics** or traits that make something **recognizable or unique**.
- In computing, it refers to **digital identity**.

11. Digital Identity:

- Includes attributes, metadata, and relationships that make an object identifiable.
- Must be **unique** (even if trivially) — identified by a unique ID or attribute.

12. Identity in Context:

- Every identity exists in a **specific context**, like a network, database, or organization.

13. Programming/Database relation:

- Programmers see identity as an object with attributes and relationships.
- Databases store this using **tables, rows, columns, and relationships**.
- This fits with **Object Role Modeling (ORM)** — identifying unique entities in data.

14. Extension to Social Profiles:

- Social media profiles (like Facebook) are digital identities too — examples of **IDaaS in action** at a global scale.
-

◆ Components of an Identity

15. An identity can belong to a **person** and may include:

- **Things you are:**
 - Biological traits — age, gender, appearance, etc.
 - **Things you know:**
 - Personal data — SSN, PINs, school info, passwords.
 - **Things you have:**
 - Fingerprints, eye patterns, bank cards, physical keys, etc.
 - **Things you relate to:**
 - Family, friends, social accounts, values, habits, preferences, etc.
-

◆ Authentication Methods

16. **Single-factor authentication:**

- Only **one proof of identity**, e.g., username + password (something you know).

17. **Two-factor authentication:**

- Combines **two proofs**, e.g., password (something you know) + token key (something you have).

18. **Multi-factor authentication (MFA):**

- Uses **three or more factors**, e.g., password + token + fingerprint (something you are).
- Ensures highest level of security.

19. **Use in Cloud:**

- MFA depends on **network security or trust services**, which are often implemented as **cloud-based IDaaS**.
-

◆ Digital Identity for Machines and Devices

20. **Not just for people:**

- Machines, devices, and software also have digital identities.

21. Where identities are stored:

- In **domain security databases, directory services, or federated data stores.**

22. Device identity:

- Each network device has a unique **MAC (Media Access Control)** address or **Ethernet Hardware Address (EHA)**.
 - The MAC address lets the network find and identify each system.
-

◆ **Microsoft Example – Windows Product Activation**

23. Windows Activation:

- Microsoft validates Windows/Office installations through **Windows Product Activation (WPA)** — creating an **identity profile** of the system.

24. Data collected during activation:

- 25-character product key & ID
- Global Unique Identifier (GUID)
- PC manufacturer
- CPU type & serial number
- BIOS checksum
- Network adapter & MAC address
- Display adapter
- SCSI/IDE adapters
- RAM amount
- Hard drive & volume serial number
- Optical drive
- Region, language, and user locale

25. How it works:

- These details are **combined and weighted** to form a unique activation code.
- If major hardware changes occur (CPU, NIC, RAM, etc.), Windows asks for **reactivation**.

26. Why it's important:

- Ensures that each Windows copy runs on authorized hardware.
- Used also for **Windows Genuine Advantage (WGA)** program.

27. Relation to Cloud:

- Both WPA and WGA are **cloud-based services** that manage digital identities — examples of **proprietary IDaaS** by Microsoft.

28. Debate:

- Some argue whether such proprietary systems should truly be called “services,” but technically, they function as **cloud identity services**.

✅ In summary:

- **IDaaS = Cloud-based identity verification and management.**
- It simplifies authentication, ensures security, supports MFA, and can apply to both users and devices.
- Examples include **Gmail login systems, Facebook profiles, corporate single sign-on (SSO), and Microsoft product activation.**

Networked Identity Service Classes – Overview

1. Purpose:

- These services are used to **validate** the identity of:
 - Websites
 - Transactions
 - Transaction participants (users)
 - Clients
 - Network services

2. How it Works:

- Different **forms of identity services** are deployed across networks to confirm that entities are who they claim to be.
- Examples include:
 - **Ticket or token services** (like Kerberos tickets or OAuth tokens)
 - **Certificate servers** (like SSL/TLS certificates)
 - **Other trust mechanisms** (digital signatures, encryption keys, etc.)

3. Cloud Integration:

- These identity mechanisms can be **moved from private networks to the cloud**, allowing centralized, scalable management through **Identity as a Service (IDaaS)**.
-

Importance and Complexity of Identity Protection

4. Expensive and Complex:

- **Identity protection** is one of the most challenging and costly parts of network computing.
- This is because multiple systems and people constantly request or verify identities.

5. Who Requests Identity Data:

- Personnel such as **HR, managers, and administrators** request identity data.
- **Systems and network resources** also verify identities for access control.

6. Heavy Network Usage:

- A **large portion of total network traffic** is related to **identity verification** and access authorization.
- Hundreds of identity-related messages are sent across the network **every minute**.

7. Ethernet Packets and Identity:

- Every **Ethernet packet** includes **header fields** that contain identifying information about its source and destination.

8. Need for Expertise:

- As networks and systems become more specialized, it's getting **harder to find qualified security experts** to manage complex identity services.

9. Value of Cloud ID Services:

- **Cloud-based or managed identity services (IDaaS)** are now seen as one of the **most valuable and cost-effective** services organizations can subscribe to.
- They reduce complexity and improve security management.

IDaaS – Identity Service Components

10. IDaaS may include any of the following service classes:

- **1. Authentication Services (Identity Verification):**
Confirms the identity of users or systems (e.g., login verification, token validation).
- **2. Directory Services:**
Centralized repositories (like LDAP or Active Directory) that store identity information and credentials.
- **3. Federated Identity:**
Allows users to use **one digital identity across multiple systems or organizations** (e.g., Google or Facebook login used on other sites).
- **4. Identity Governance:**
Defines policies for **who can access what**, ensuring compliance and accountability.

- **5. Identity and Profile Management:**
Handles **user profiles**, roles, permissions, and personal details.
 - **6. Policies, Roles, and Enforcement:**
Defines rules for access, sets user **roles**, and enforces security policies automatically.
 - **7. Provisioning (External Policy Administration):**
Controls **creation, modification, and deletion of user accounts** based on policies.
 - **8. Registration:**
Process by which new users or entities are **added** to the system and assigned credentials.
 - **9. Risk and Event Monitoring (Including Audits):**
Tracks login attempts, access patterns, and suspicious behavior to detect **security risks**.
Also supports **auditing** for compliance.
 - **10. Single Sign-On (SSO) Services:**
Allows users to **log in once** and gain access to **multiple applications or systems** without re-entering credentials each time.
-

Security and Regulation Considerations

11. Data Sharing and Privacy:

- Sharing identity-related data across networks is **highly regulated** by governments and data protection laws (e.g., GDPR).
- Access to identity data must be **limited only to authorized entities**.

12. Minimal Data Principle:

- Only the **minimum amount of information** necessary for validation should be shared.
- This ensures **privacy and security** are maintained.

13. Identity Relationship:

- The **level of access** granted to a user or system defines an **identity relationship** — i.e., how much identity information one entity is allowed to see or use about another.
-

In summary:

- **Networked identity services** verify and manage digital identities across networks and cloud systems.
- They include components like **authentication, directory, governance, SSO, and risk monitoring**.
- Because identity management is complex, **cloud-based IDaaS** offers a **secure, cost-effective, and centralized** solution for modern enterprises.

Identity System Codes of Conduct

Certain codes of conduct must be observed legally, and if not legally at the moment, then certainly on a moral basis. Cloud

computing services that don't observe these codes do so at their peril. In working with IDaaS software, evaluate IDaaS

applications on the following basis:

- User control for consent: Users control their identity and must consent to the use of their information.
- **Minimal Disclosure**: The minimal amount of information should be disclosed for an Intended use.
- Justifiable access: Only parties who have a justified use of the information contained in a digital identity and have a trusted

identity relationship with the owner of the information may be given access to that information.

- Directional Exposure: An ID system must support bidirectional identification for a public entity so that it is discoverable

and a unidirectional identifier for private entities, thus protecting the private ID.

- Interoperability: A cloud computing ID system must interoperate with other identity services from other identity providers.

- Unambiguous human identification: An IDaaS application must provide an unambiguous mechanism for allowing a human

to interact with a system while protecting that use against an identity attack.

- Consistency of Service: An IDaaS service must be simple to use, consistent across all its uses, and able to operate in

different contexts using different technologies

IDaaS Interoperability – Overview

1. Meaning:

- **Identity as a Service (IDaaS)** allows easy integration of identity management into applications **without heavy coding**.
- The **identity logic and data (user attributes)** are maintained **externally** — not inside each app — making it simpler and more secure.

2. Purpose:

- IDaaS enables **different systems and applications** to work together for authentication and authorization.

- This is achieved through **interoperability** — i.e., using common **industry standards** that all ID systems understand.

3. Why Interoperability Matters:

- Cloud environments often use **different technologies**.
 - To ensure seamless communication, IDaaS must follow **standardized protocols** for **identity verification, authorization, and auditing**.
-

Key Standards That Enable IDaaS Interoperability

◆ 1. User-Centric Authentication

4. Purpose:

- Gives users control over their identity and credentials when logging into multiple systems.

5. Common Standards:

- **OpenID** – Allows users to log in to multiple sites using a single identity (e.g., “Login with Google”).
- **Microsoft CardSpace** – Uses digital “information cards” for secure identity exchange.

6. How It Helps:

- Simplifies user login and allows **cross-platform identity verification** without repeatedly creating new accounts.
-

◆ 2. XACML (eXtensible Access Control Markup Language)

7. Purpose:

- A **general-purpose authorization policy language** used to define and enforce **access control policies** in distributed systems.

8. Function:

- XACML checks whether a user or system **should be allowed or denied** access based on predefined rules.

9. Integration with SAML:

- XACML works with **SAML (Security Assertion Markup Language)**.
- **SAML** sends the access request → **XACML** checks it against policies → allows or denies the action.

10. Use Case Example:

- When a user tries to access a document in the cloud, SAML sends a request and XACML validates it against authorization rules.
-

◆ 3. SPML (Service Provisioning Markup Language)

11. Purpose:

- An **XML-based language** used for **service provisioning** — creating, modifying, or deleting user accounts and resources across systems.

12. Origin:

- A standard developed by **OASIS** (Organization for the Advancement of Structured Information Standards) under its **Provisioning Services Technical Committee (PSTC)**.

13. Role in Cloud:

- Ensures different cloud services can **communicate and manage user provisioning requests** automatically.
 - Conforms to **Service-Oriented Architecture (SOA)** principles.
-

◆ 4. XDAS (eXtensible Distributed Audit Service)

14. Purpose:

- Provides **accountability and auditing** for user activities within a system.

15. Functions:

- Tracks **who accessed what and when**.
- Detects **security policy violations** when unauthorized access attempts occur.
- Helps maintain **compliance and forensic tracking**.

16. Why It's Important:

- Ensures that **every identity-based action** is recorded for auditing and security assurance.
-

◆ Identity Governance Framework (IGF)

17. Definition:

- IGF is a **standards initiative** from the **Liberty Alliance** focused on the **exchange, control, and governance** of identity information across systems.

18. Liberty Alliance Overview:

- Founded in **2001** by a group of major technology companies.

- Its goal: to **promote open identity exchange** through **policy-based standards** that maintain **privacy and security**.

19. Supported Standards:

- IGF uses technologies such as:
 - **WS-Trust** (Web Services Trust Language)
 - **ID-WSF** (Identity Web Services Framework)
 - **SAML** (Security Assertion Markup Language)
 - **LDAP** (Lightweight Directory Access Protocol)

20. Main Objectives of IGF:

- Ensure **privacy control** during identity exchange.
 - Enable **auditing and enforcement** of privacy policies.
 - Facilitate **secure interoperability** between organizations.
-

◆ Liberty Alliance Contributions

21. 2009 Developments:

- Released two key components:
 1. **CARML (Client Attribute Requirements Markup Language):**
 - Defines what client attributes (like username, email, etc.) an application can request or use.
 2. **IGF Privacy Constraints:**
 - Defines how identity data can be shared securely while maintaining privacy.

22. Resulting Project – Aristotle:

- An **open-source project** launched by the Liberty Alliance.
 - Goal: create an **API for identity management** using IGF principles.
 - Hosted at: openliberty.org/wiki/index.php/ProjectAris
-

✓ Summary – Why IDaaS Interoperability Is Important

23. Simplifies Development:

- Applications can easily integrate identity management without rebuilding login or policy logic.

24. Ensures Compatibility:

- Common standards (OpenID, SAML, XACML, SPML, XDAS, IGF) ensure **different systems work together** securely.

25. Improves Security & Compliance:

- Enables **trust-based access**, **policy enforcement**, and **auditing** across organizations.

26. Promotes Privacy:

- Frameworks like **IGF** ensure user data and identity attributes are **shared responsibly** under **strict privacy controls**.
-

User Authentication (Simplified Point-Wise Explanation)

OpenID

- 1. Definition:**
OpenID is an industry standard for authenticating users ("end users") by storing their digital identity in a common format.
- 2. Storage of Identity:**
The identity is stored in the system of any OpenID service provider and translated into a unique identifier.
- 3. Identifier Format:**
Identifiers are in the form of a URL or an Extensible Resource Identifier (XRI), authenticated by the OpenID provider.
- 4. Compatibility:**
Any software application that follows the OpenID standard can accept and verify an OpenID authenticated by a trusted provider.
- 5. Examples of OpenID Providers:**
AOL, Facebook, Google, IBM, Microsoft, MySpace, Orange, PayPal, VeriSign, LiveJournal, Ustream, Yahoo!, and others.
- 6. Unique URL Identity:**
The OpenID standard applies to the unique identity (URL), and the service provider decides how authentication is handled.
- 7. Forms of Authentication:**
Authentication can include passwords, smart cards, hardware keys, tokens, and biometrics.
- 8. Support Organization:**
OpenID is supported by the OpenID Foundation (<http://openid.net/foundation/>), a non-profit promoting the technology.

Examples of Trusted Providers and URL Formats

1. Blogger: <username>.blogger.com or <blogid>.blogspot.com
2. MySpace: myspace.com/<username>
3. Google: <https://www.google.com/accounts/o8/id>
4. Google Profile: google.com/profiles/<username>
5. Microsoft: accountservices.passport.net/
6. MyOpenID: <username>.myopenid.com
7. Orange: openid.orange.fr/username or orange.fr/
8. VeriSign: <username>.pip.verisignlabs.com
9. WordPress: <username>.wordpress.com
10. Yahoo!: openid.yahoo.com

Login Process

1. After logging into a trusted OpenID provider, you can access other websites that support OpenID.
 2. When you try to access a site through your browser (user-agent), that site acts as a “relying party.”
 3. The relying party requests the server (or server-agent) to verify the user’s identifier.
-

CardSpace

1. Definition:
CardSpace is a Microsoft software client that is part of the company’s Identity Metasystem, used for cloud authentication.
2. Technology Base:
Built on OASIS standards — WS-Trust, WS-Security, WS-SecurityPolicy, and WS-MetadataExchange.
3. Interoperability:
Any application that follows these OASIS WS standards can interoperate with CardSpace.
4. Availability:
Introduced with .NET Framework 3.0, works on Windows XP, Server 2003, and later.
Installed by default on Windows Vista and Windows 7.
5. Authentication Mechanism:
 - An Information Card can be requested using an HTML <OBJECT> tag.
 - The trusted Identity Provider creates an encrypted and digitally signed token using the Security Token Service (STS).
 - This follows a WS-Trust request/reply mechanism.

6. Alternative Use:

CardSpace can be used as an alternative to OpenID and SAML, and also supports Windows Live ID accounts.

IDaaS Interoperability = using **standard identity languages and frameworks** (OpenID, SAML, XACML, SPML, XDAS, IGF)

→ to make sure **authentication, authorization, and identity exchange** work **smoothly, securely, and consistently** across cloud platforms.

Authorization Markup Languages (Simplified Point-Wise Explanation)

General Information

1. In **cloud computing**, most information **requests and replies** are in the form of **XML** files.
2. **XML files** are **text files** and **self-describing**, meaning they contain or point to a **schema** that defines the data structure.
3. Within the identity framework, two important XML-based languages are:
 - **XACML** (eXtensible Access Control Markup Language)
 - **SAML** (Security Assertion Markup Language)

XACML (eXtensible Access Control Markup Language)

4. **XACML** is an **OASIS standard** for defining **policy statements** in XML format to support **authentication processes**.
5. A **policy** in XACML describes:
 - A **Subject element** that requests an action
 - An **Action element** that represents the operation being requested
 - A **Resource** (like a service, system component, or data) on which the action is to be performed
6. These three elements — **Subject**, **Action**, and **Resource** — operate within an **Environment**, which can also be defined by an **Action element**.
7. **Subject** and **Action** elements can have **one or more attributes**.
8. **Resources** usually have a **single attribute**, which is typically their **URL**.

SAML (Security Assertion Markup Language)

9. A **SAML assertion** is a **security statement** written in XML that makes a claim about **authentication, attributes, or authorization**.
 10. The statement generally follows this form:
“**Assertion X created at Time T by User U about Subject S is true when Conditions C are TRUE.**”
 11. It is the responsibility of the **identity provider** to **parse** and **validate** this statement.
 12. The **SAML protocol request** is also called a **query**, and it supports three types:
 - **Authentication query**
 - **Attribute query**
 - **Authorization decision query**
 13. **SAML requests** use a **SOAP binding**, meaning they are **embedded in a SOAP wrapper** within an **HTTP message**.
 14. **SAML** is used to enable **Web Browser Single Sign-On (SSO)**, where the **web browser** acts as the **user agent** requesting access to a resource authorized by a **SAML service provider**.
 15. The **service provider** receives the user’s access request and sends an **authentication request** to the **SAML identity provider** directly through the web browser (user agent).
 16. This mechanism is shown in the **SAML Single Sign-On Request/Response** model.
-

SPML (Service Provisioning Markup Language)

17. **SPML** is another **OASIS open standard** designed for **service provisioning** in cloud systems.
18. **Provisioning** means preparing, reserving, accessing, using, and releasing a **resource** after the transaction completes.
 - Example: Reserving and using a **phone line** or a **Virtual Private Network (VPN)**.
19. A **provisioning system** consists of **three main components**:
 - **Requesting Authority (RA)**: The client that sends the request.
 - **Provisioning Service Point (PSP)**: The cloud component that receives the request and returns a response.
 - **Provisioning Service Target (PST)**: The software application where the provisioning action is performed.
20. The **SPML system** acts as an **architectural layer**, ensuring that **identity information** needs to be entered **only once** across these three components.
21. **SPML** is used to:
 - Prepare **web services** and **applications** for use.
 - Signal when a resource is **ready for use**.

- Signal when a **transaction or use** is **completed**.
22. Using **SPML**, a system can:
- Provide **automated user and system access**.
 - **Enforce access rights**.
 - Make **cloud computing services** available **across network systems**.
23. Without a **provisioning system**, a **cloud computing system** becomes **inefficient**.

Compliance as a Service (CaaS) — Simplified Point-Wise Explanation

Definition

1. **Compliance as a Service (CaaS)** is a **cloud-based model** that helps organizations meet **regulatory compliance requirements**.
 2. It does this by **outsourcing compliance management** to a **third-party provider**.
 3. CaaS provides **compliance implementation, management, and maintenance services** to companies in **regulated industries** such as **healthcare, finance, and government**.
-

Nature and Challenges

4. **Cloud computing** naturally operates across **multiple jurisdictions**, meaning:
 - The **laws** of the **country of origin** of a request may differ from the **laws** of the **country where it is processed**.
 - Both may also differ from the **laws** of the **country where the service is provided**.
 5. **Compliance** is **not just about access control** or assigning service tokens — it is a **complex process** that requires **expertise**.
 6. Although **CaaS** is often discussed, **few actual implementations** exist as a **general product** in cloud computing.
-

Architecture and Functionality

7. A **CaaS application** would need to act as a **trusted third party**, since it functions as a **man-in-the-middle service** between users and providers.
8. It may need to be **architected as a separate layer** within a **Service-Oriented Architecture (SOA)** to ensure **trust**.
9. A **CaaS system** must be capable of:
 - Managing **cloud relationships**.

- Understanding and enforcing **security policies and procedures**.
 - Handling **information and privacy** properly.
 - Being **aware of geography and jurisdictional laws**.
 - Providing **incident response** mechanisms.
 - Supporting **data archiving** and **querying**.
 - Operating under a clearly defined **Service Level Agreement (SLA)**.
10. Overall, **CaaS** has the potential to be a **highly valuable, value-added service** for organizations.
-

Implementation and Examples

11. To implement **CaaS**, companies are developing “**vertical clouds**” — cloud platforms that focus on **specific industries**.
12. Examples of **vertical clouds** offering **CaaS capabilities** include:
- **athenahealth** (medical industry) — athenahealth.com
 - **bankserv** (banking industry) — bankserv.com
 - **ClearPoint PCI Compliance-as-a-Service** — for **merchant transactions** under **Payment Card Industry Data Security Standard (PCI DSS)**.
 - **FedCloud** (government sector) — fedcloud.com
 - **Rackserve PCI Compliant Cloud** — rackspace.com
-

Private Cloud Implementation

13. A **CaaS system** can also be built **inside a private cloud**, where:
- **Data remains under the control** of a single entity.
 - Ensures **secure control** and allows **auditing of transactions**.
14. **Most existing compliance systems** in cloud computing are **built using private clouds** for **better control and security**.
-

Value and Potential

15. A **well-implemented CaaS service** can:
- **Measure risks** involved in maintaining compliance.
 - **Ensure** or **indemnify** customers against those risks.
16. CaaS could also be used to **guarantee that e-mails conform to certain standards**, which could:

- Become a **new electronic postal service** for national networks.
- Help **reduce or eliminate spam threats**.

✅ **In summary:** CaaS is a **trusted, specialized cloud service** that ensures organizations meet legal and industry-specific compliance requirements by managing regulations, privacy, auditing, and risk — often through **vertical or private clouds**.